Dataset Distillation by Matching Training Trajectories

George Cazenavette[1] Tongzhou Wang[2] Antonio Torralba[2] Alexei Efros[3] Jun-Yan Zhu[1]

[1]Carnegie Mellon University [2]Massachusetts Institute of Technology [3]UC Berkeley



Figure 1. Example distilled images from 32x32 CIFAR-100 (top), 64x64 Tiny ImageNet (middle), and 128x128 ImageNet subsets (bottom). Training a standard CNN using only such distilled images (as few as one per category) yields a trained model capable of test accuracy significantly better than previous methods of dataset distillation.

Abstract

Dataset distillation is the task of synthesizing a small dataset such that a model trained on the synthetic set will match the test accuracy of the model trained on the full dataset. The task is extremely challenging as it often involves backpropagating through the full training process or assuming the strong constraint that a single training step on distilled data can only imitate a single step on real data. In this paper, we propose a new formulation that optimizes our distilled data to guide networks to a similar state as those trained on real data across many training steps. Given a network, we train it for several iterations on our distilled data and optimize the distilled data with respect to the distance between the synthetically trained parameters and the parameters trained on real data. To efficiently obtain the initial and target network parameters for large-scale datasets, we pre-compute and store training trajectories of expert networks trained on the real dataset. Our method handily outperforms existing methods and also allows us to distill with higher-resolution visual data.

1. Introduction

In the seminal 2015 paper, Hinton et al. [15] proposed *model distillation*, which aims to distill the knowledge of a complex model into a simpler one. *Dataset distillation*, proposed by Wang et al. [43], is a related but orthogonal task: rather than distilling the model, the idea is to distill the dataset. As shown in Figure 2, the goal is to distill the knowledge from a large training dataset into a very small set of synthetic training images (as low as one image per class) such that training a model on the distilled data would give a similar test performance as training one on the original dataset. Dataset distillation has become a lively research topic in machine learning [2, 25, 26, 38, 44, 45, 46] with various applications, such as continual learning, neural architecture search, and privacy-preserving ML. Still, the problem has so far been of mainly theoretical interest, since most prior methods focus on toy datasets, like MNIST and CIFAR, while struggling on real, high-resolution images. In this work, we present a new approach to dataset distillation that not only outperforms previous work in performance, but is also applicable to large-scale datasets, as shown in Figure 1.

Unlike classical data compression, dataset distillation

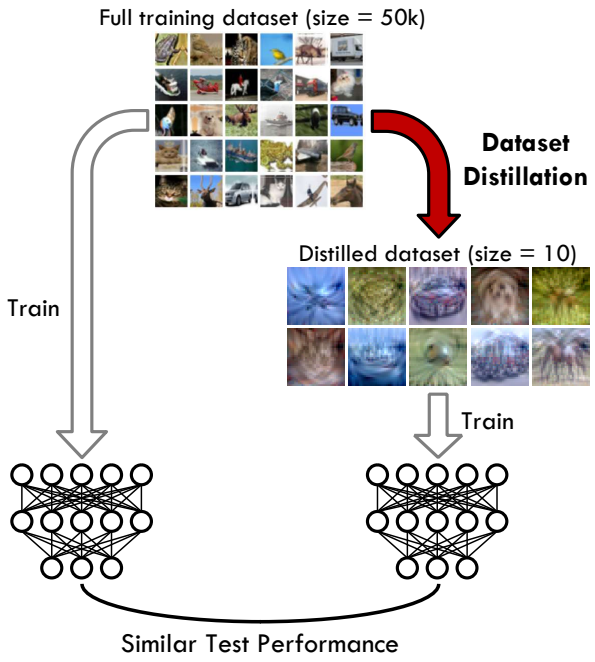


Figure 2. Dataset distillation aims to generate a small synthetic dataset for which a model trained on it can achieve similar test performance as a model trained on the whole real train set.

aims for a small synthetic dataset that still exhibits adequate task-related information so that models trained on it can generalize to unseen test data, as shown in Figure 2. Thus, the distilling algorithm must strike a delicate balance by heavily compressing information without completely obliterating the discriminative features. Several methods consider full training trajectories [25, 26, 43] but often require huge compute and memory and suffer from inexact relaxation [25, 26] or training instability of unrolling many iterations [23, 43]. To reduce the optimization difficulty, other methods [44, 46] focus on short-range behavior, enforcing a single training step on distilled data to match that on real data. However, error may accumulate in evaluation, where the distilled data is applied over many steps. We confirm this hypothesis experimentally in Section 4.2.

To address the above challenges, we sought to directly imitate the long-range training dynamics of networks trained on real datasets. In particular, we match segments of parameter trajectories trained on synthetic data with segments of pre-recorded trajectories from models trained on real data and thus avoid being short-sighted (i.e., focusing on single steps) or difficult to optimize (i.e., modeling the full trajectories). Treating the real dataset as the gold standard for guiding the network's training dynamics, we can consider the induced sequence of network parameters to be an *expert trajectory*. If our distilled dataset were to induce a network's training dynamics to follow these expert trajectories, then the synthetically trained network would land at a place close to the model trained on real data (in the parameter space) and achieve similar test performance.

In our method, our loss function *directly* encourages the distilled dataset to guide the network optimization along a similar trajectory (Figure 3). To do this, we first train a set of models from scratch on the real dataset and record their expert training trajectories. We then initialize a new model with a random time step from a randomly chosen expert trajectory and train for several iterations on the *synthetic* dataset. Finally, we penalize the distilled data based on how far this synthetically trained network deviated from the expert trajectory and back-propagate through the training iterations. Essentially, we transfer the knowledge from many expert training trajectories to the distilled images.

Extensive experiments show that our method handily outperforms existing dataset distillation methods as well as coreset selection methods on standard datasets, including CIFAR-10, CIFAR-100, and Tiny ImageNet. For example, we achieve 46.3% with a *single* image per class and 71.5% with 50 images per class on CIFAR-10, compared to the previous state of the art (28.8% / 63.0% from [44, 45] and 36.1% / 46.5% from [26]). Furthermore, our method also generalizes well to larger data, allowing us to see high 128×128-resolution images distilled from ImageNet [6] for the first time. Finally, we analyze our method through additional ablation studies and visualizations. Our code and models will be available upon publication.

2. Related Work

Dataset Distillation. Dataset distillation was first introduced by Wang et al. [43], who proposed expressing the model weights as a function of distilled images and optimized them using gradient-based hyperparameter optimization [23], which is also widely used in meta-learning research [8, 27]. Subsequently, several works significantly improved the results by learning soft labels [2, 38], amplifying learning signal via gradient matching [46], adopting augmentations [44], and optimizing with respect to the infinite-width kernel limit [25, 26]. Dataset distillation has enabled various applications including continual learning [43, 44, 46], efficient neural architecture search [44, 46], federated learning [11, 37, 49], and privacy-preserving ML [22, 37] for images, text, and medical imaging data. As mentioned in the introduction, our method does not rely on single-step behavior matching [44, 46], costly unrolling of full optimization trajectories [38, 43], or large-scale Neural Tangent Kernel computation [25, 26]. Instead, our method achieves long-range trajectory matching by transferring the knowledge from pre-trained experts.

Concurrent with our work, a method of Zhao and Bilen [45] completely disregards optimization steps, instead focusing on distribution matching between synthetic and real data. While this method is applicable to higher-resolution

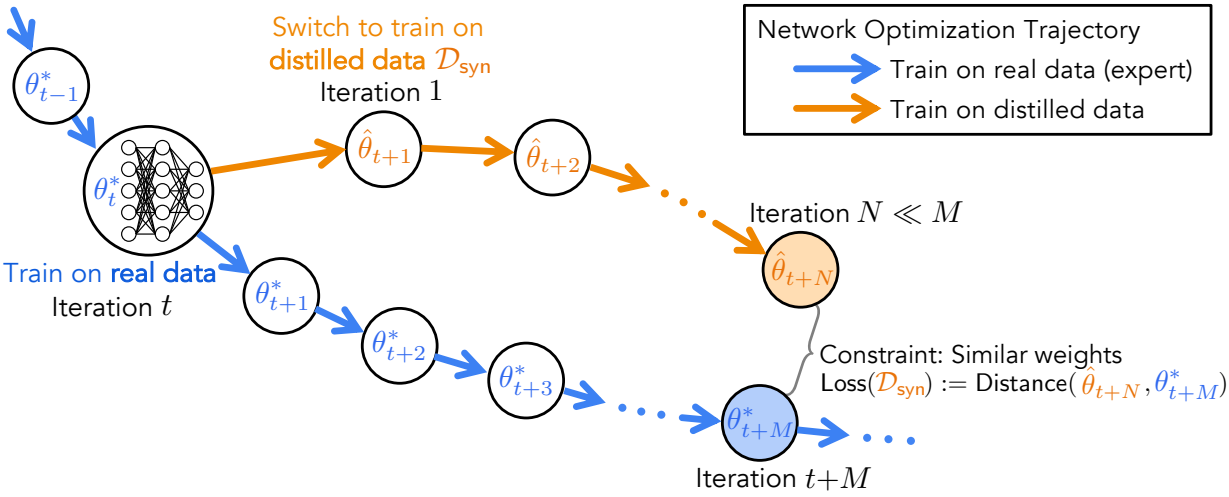


Figure 3. We perform long-range parameter matching between training on distilled synthetic data and training on real data. Starting from the same initial parameters, we train distilled data $\mathcal{D}_{\text{syn}}$ such that N training steps on them match the same result (in parameter space) from much more M steps on real data.

datasets (e.g., Tiny ImageNet) with the reduced memory requirements, it attains inferior performance in most cases (e.g., when compared to previous works [44, 46]). In contrast, our method simultaneously reduces memory costs while outperforming existing works [44, 46] and the concurrent method [45] on both standard benchmarks and higher-resolution datasets.

A related line of research learns a generative model to synthesize training data [24, 36]. However, such methods do not generate a small-size dataset, and are thus not comparable with our and other dataset distillation works.

Imitation Learning. Imitation learning attempts to learn a good policy by observing a collection of expert demonstrations [29, 30, 31]. Behavior cloning trains the learning policy to act the same as expert demonstrations. Some more sophisticated formulations involve on-policy learning with labelling from the expert [33], while other approaches avoid any label at all, e.g., via distribution matching [16]. Such methods (behavior cloning in particular) have shown to achieve good performance in offline settings [9, 12]. Our method can be viewed as imitating a collection of expert network training trajectories, which are obtained via training on real datasets. Therefore, it can be considered as doing imitation learning over optimization trajectories.

Coreset and Instance Selection. Similar to dataset distillation, coreset [1, 4, 13, 34, 41] and instance selection [28] aim to select a subset of the entire training dataset, where training on this small subset achieves good performance. Most of such methods do not apply to modern deep learning, but new formulations based on bi-level optimization have shown promising results on applications like continual learning [3]. Related to coreset, other lines of research aim to understand which training samples are "valuable" for modern machine learning, including measuring single-example accuracy [20] and counting mis-classification rates [39]. In fact, dataset distillation is a generalization of such ideas, as the distilled data do not need to be realistic or come from the training set.

3. Method

Dataset Distillation refers to the curation of a small, synthetic training set $\mathcal{D}_{\text{syn}}$ such that a model trained on this synthetic data will have similar performance on the real test set as a model trained on the large, real training set $\mathcal{D}_{\text{real}}$. In this section, we describe our method that directly mimics long-range behavior of real-data training, matching multiple training steps on distilled data to *many* more steps on the real data.

In Section 3.1, we discuss how we obtain expert trajectories of networks trained on real datasets. In Section 3.2, we describe a new dataset distillation method that explicitly encourages the distilled dataset to induce similar long-range network parameter trajectories as the real dataset, resulting in a synthetically-trained network that performs similarly to a network trained on real data. Finally, Section 3.3 describes our techniques to reduce memory consumption.

3.1. Expert Trajectories

The core of our method involves using *expert trajectories* τ^* to guide the distillation of our synthetic dataset. By expert trajectories, we mean the time sequence of parameters $\{\theta^*_t\}_0^T$ obtained during the training of a neural network on the *full, real dataset*. To generate these expert trajectories, we simply train a large number of networks on the real dataset and save their snapshot parameters at every epoch. We call these sequences of parameters "expert trajectories" because they represent the theoretical upper bound for the dataset distillation task: the performance of a network trained on the

Algorithm 1 Dataset Distillation via Trajectory Matching

Input: $\{\tau_i^*\}$: set of expert parameter trajectories trained on $\mathcal{D}_{\text{real}}$.
Input: M: # of updates between starting and target expert params.
Input: N: # of updates to student network per distillation step.
Input: $\mathcal{A}$: Differentiable augmentation function.
Input: $T^+ < T$: Maximum start epoch.

Initialize distilled data $\mathcal{D}_{\text{syn}} \sim \mathcal{D}_{\text{real}}$
Initialize trainable learning rate $\alpha := \alpha_0$ for apply $\mathcal{D}_{\text{syn}}$
for each distillation step... **do**
  ▷ Sample expert trajectory: $\tau^* \sim \{\tau_i^*\}$ with $\tau^* = \{\theta_t^*\}_0^T$
  ▷ Choose random start epoch, $t \leq T^+$
  ▷ Initialize student network with expert params:
    $\hat{\theta}_t := \theta_t^*$
  for $n = 0 \rightarrow N-1$ **do**
    ▷ Sample a mini-batch of distilled images:
      $b_{t+n} \sim \mathcal{D}_{\text{syn}}$
    ▷ Update student network w.r.t. classification loss:
      $\hat{\theta}_{t+n+1} = \hat{\theta}_{t+n} - \alpha \nabla \ell(\mathcal{A}(b_{t+n}); \hat{\theta}_{t+n})$
  end for
  ▷ Compute loss between ending student and expert params:
    $\mathcal{L} = \|\hat{\theta}_{t+N} - \theta_{t+M}^*\|_2^2 \;/\; \|\theta_t^* - \theta_{t+M}^*\|_2^2$
  ▷ Update $\mathcal{D}_{\text{syn}}$ and α with respect to $\mathcal{L}$
end for

Output: distilled data $\mathcal{D}_{\text{syn}}$ and learning rate α

full, real dataset. Similarly, we define student parameters $\hat{\theta}_t$ as the network parameters trained on synthetic images at the training step t. Our goal is to distill a dataset that will induce a similar trajectory (given the same starting point) as that induced by the real training set such that we end up with a similar model.

Since these expert trajectories are computed using only real data, we can pre-compute them before distillation. All of our experiments for a given dataset were performed using the same pre-computed set of expert trajectories, allowing for rapid distillation and experimentation.

3.2. Long-Range Parameter Matching

Our distillation process learns from the generated sequences of parameters making up our expert trajectories $\{\theta_t^*\}_0^T$. Unlike previous work, our method directly encourages the long-range training dynamics induced by our synthetic dataset to match those of networks trained on the real data.

At each distillation step, we first sample parameters from one of our expert trajectories at a random timestep θ_t^* and use these to initialize our student parameters $\hat{\theta}_t := \theta_t^*$. Placing an upper bound T^+ on t lets us ignore the less informative later parts of the expert trajectories where the parameters do not change much.

With our student network initialized, we then perform N gradient descent updates on the student parameters with respect to the classification loss of the *synthetic* data:

$$\hat{\theta}_{t+n+1} = \hat{\theta}_{t+n} - \alpha \nabla \ell(\mathcal{A}(\mathcal{D}_{\text{syn}}); \hat{\theta}_{t+n}), \tag{1}$$

where $\mathcal{A}$ is the differentiable augmentation technique [17, 40, 47, 48] used in previous work [44], and α is the (trainable) learning rate used to update the student network. Any data augmentation used during distillation must be differentiable so that we can back-propagate through the augmentation layer to our synthetic data. Our method does not use differentiable *Siamese* augmentation since there is no real data used during the distillation process; we are only applying the augmentations to synthetic data at this time. However, we do use the same types of differentiable augmentations on real data during the generation of the expert trajectories.

From this point, we return to our expert trajectory and retrieve the expert parameters from M training updates after those used to initialize the student network θ_{t+M}^*. Finally, we update our distilled images according to the weight matching loss: i.e., the normalized squared L_2 error between the updated student parameters $\hat{\theta}_{t+N}$ and the known future expert parameters θ_{t+M}^*:

$$\mathcal{L} = \frac{\|\hat{\theta}_{t+N} - \theta_{t+M}^*\|_2^2}{\|\theta_t^* - \theta_{t+M}^*\|_2^2}, \tag{2}$$

where we normalize the L_2 error by the expert distance traveled so that we still get a strong signal from later training epochs where the expert does not move as much. This normalization also helps self-calibrate the magnitude difference across neurons and layers. We have also experimented with other choices of loss functions such as a cosine distance, but find our simple L_2 loss works better empirically. We also tried to match the network's output logits between expert trajectory and student network but did not see a clear improvement. We speculate that backpropagating from the network output to the weights introduces additional optimization difficulty.

We then minimize this objective to update the pixels of our distilled dataset, along with our trainable learning rate α, by back-propagating through all N updates to the student network. The optimization of trainable learning rate α serves as automatic adjusting for the number of student and expert updates (hyperparameters M and N). We use SGD with momentum to optimize $\mathcal{D}_{\text{syn}}$ and α with respect to the above objective. Algorithm 1 illustrates our main algorithm.

3.3. Memory Constraints

Given that we are back-propagating through many gradient descent updates, memory consumption quickly becomes an issue when our distilled dataset is sufficiently large, as we have to jointly optimize all the images of all the classes at each optimization step. To reduce memory consumption and ease the learning problem, previous methods distill one class

	Img/Cls	Ratio %	Coreset Selection			Training Set Synthesis						Full Dataset
			Random	Herding	Forgetting	DD† [43]	LD† [2]	DC [46]	DSA [44]	DM [45]	Ours	
CIFAR-10	1	0.02	14.4 ± 2.0	21.5 ± 1.2	13.5 ± 1.2	-	25.7 ± 0.7	28.3 ± 0.5	28.8 ± 0.7	26.0 ± 0.8	**46.3 ± 0.8**	84.8 ± 0.1
	10	0.2	26.0 ± 1.2	31.6 ± 0.7	23.3 ± 1.0	36.8 ± 1.2	38.3 ± 0.4	44.9 ± 0.5	52.1 ± 0.5	48.9 ± 0.6	**65.3 ± 0.7**	
	50	1	43.4 ± 1.0	40.4 ± 0.6	23.3 ± 1.1	-	42.5 ± 0.4	53.9 ± 0.5	60.6 ± 0.5	63.0 ± 0.4	**71.5 ± 0.3**	
CIFAR-100	1	0.2	4.2 ± 0.3	8.4 ± 0.3	4.5 ± 0.2	-	11.5 ± 0.4	12.8 ± 0.3	13.9 ± 0.3	11.4 ± 0.3	**24.3 ± 0.3**	56.2 ± 0.3
	10	2	14.6 ± 0.5	17.3 ± 0.3	15.1 ± 0.3	-	-	25.2 ± 0.3	32.3 ± 0.3	29.7 ± 0.3	**39.4 ± 0.1**	
	50	10	30.0 ± 0.4	33.7 ± 0.5	30.5 ± 0.3	-	-	-	42.8 ± 0.4	**43.6 ± 0.4**	**43.8 ± 0.2**	
Tiny ImageNet	1	0.2	1.4 ± 0.1	2.8 ± 0.2	1.6 ± 0.1	-	-	-	-	3.9 ± 0.2	**8.8 ± 0.3**	37.6 ± 0.4
	10	2	5.0 ± 0.2	6.3 ± 0.2	5.1 ± 0.2	-	-	-	-	12.9 ± 0.4	**23.2 ± 0.2**	
	50	10	15.0 ± 0.4	16.7 ± 0.3	15.0 ± 0.3	-	-	-	-	24.1 ± 0.3	**28.0 ± 0.3**	

Table 1. Comparing finite-width network-based distillation and coreset selection methods. As in previous work, we distill the given number of images per class using the training set, train a neural network on the synthetic set, and evaluate on the test set. To get $\bar{x} \pm s$, we train 5 networks from scratch on the distilled dataset. Note that the earlier works DD† and LD† use different architectures, i.e., LeNet [21] for MNIST and AlexNet [19] for CIFAR-10. The other methods all use 128-width ConvNet for distillation and evaluation.

at a time [44, 45, 46], but this may not be an ideal strategy for our method since the expert trajectories are trained on all classes simultaneously.

We could potentially circumvent this memory constraint by sampling a new mini-batch at every distillation step (the outer loop in Line 3 of Algorithm 1). Unfortunately, this comes with its own issues, as redundant information could be distilled into multiple images across the synthetic dataset, degrading to catastrophic mode collapse in the worst case.

Instead, we can sample a new mini-batch b for every update of the *student network* (i.e., the inner loop in Line 10 of Algorithm 1) such that all distilled images will have been seen by the time the final weight matching loss (Eqn. 2) is calculated. The mini-batch b still contains images from different classes but has much fewer images per class. In this case, our student network update then becomes

$$\begin{aligned} b_{t+n} &\sim \mathcal{D}_{\text{syn}} \\ \hat{\theta}_{t+n+1} &= \hat{\theta}_{t+n} - \alpha \nabla \ell(\mathcal{A}(b_{t+n}); \hat{\theta}_{t+n}). \end{aligned} \tag{3}$$

This method of batching allows us to distill a much larger synthetic dataset while ensuring some amount of heterogeneity among the distilled images of the same class.

4. Experiments

We evaluate our method on various datasets, including

- 32×32 CIFAR-10 and CIFAR-100 (Section 4.1), two commonly used datasets in dataset distillation literature,
- 64×64 Tiny ImageNet (Section 4.3), a recent benchmark by the concurrent work [45], and
- our new 128×128 ImageNet subsets (Section 4.4).

We provide additional visualizations and ablation studies in the supplementary material.

Evaluation and Baselines. We evaluate various methods according to the standard protocol: training a randomly initialized neural network from scratch on *distilled* data and evaluating on the validation set.

To generate the distilled images for our method, we employ the distillation process detailed in the previous section and Algorithm 1, using the same suite of differentiable augmentations as done in previous work [44, 45]. The hyperparameters used for each setting (real epochs per iteration, synthetic updates per iteration, image learning rate, etc.) can be found in the supplemental material.

We compare to several recent methods including Dataset Distillation (DD) [43], Flexible Dataset Distillation (LD) [2], Dataset Condensation [46] (DC), Differentiable Siamese Augmentation [44] (DSA), a concurrent work Distribution Matching [45] (DM), and a method based on the infnite-width kernel limit [25, 26] (KIP). We also compare our methods with instance selection algorithms including random selection (random), herding methods [4] (herding), and example forgetting [39] (forgetting).

Network Architectures. Staying with precedent [26, 44, 45, 46], we mainly employ a simple ConvNet architecture designed by Gidaris and Komodakis [10] for our distillation tasks. The architecture consists of several convolutional blocks, each containing a 3×3 convolution layer with 128 filters, Instance normalization [42], RELU, and 2×2 average pooling with stride 2. After the convoluation blocks, a single linear layer produces the logits. The exact number of such blocks is decided by the dataset resolution and is specified below for each dataset. Staying with this simple architecture allows us to directly analyze the effectiveness of our core method and remain comparable with previous works.

4.1. Low-Resolution Data (32×32)

For low-resolution tasks, we begin with the 32×32 CIFAR-10 and CIFAR-100 datasets [18]. For these datasets, we employ ZCA whitening as done in previous work [25, 26], using the Kornia [32] implementation with default parameters. Staying with precedent, we use a depth-3 ConvNet taken directly from the open-source code [44, 46].

As seen in Table 1, our method significantly outperforms all baselines in every setting but one, in where we are within one standard deviation above the next best (DM). In fact, on the one image per class setting, we improve the next best method (DSA [44]) to almost twice test accuracy, on

	Img/Cls	Ratio %	KIP to NN (1024-width)	KIP to NN (128-width)	Ours (128-width)
CIFAR-10	1	0.02	49.9	36.1	**46.3**
	10	0.1	62.7	46.5	**65.3**
	50	1	68.6	54.1	**71.5**
CIFAR-100	1	0.2	15.7	3.3	**24.3**
	10	2	28.3	14.3	**39.4**

Table 2. Kernel Inducing Point (KIP) [26] performs distillation using the infinite-width network limit. We consistently outperform KIP when evaluating on the same finite-width network, and almost always outperform KIP applied on wider networks.

		Evaluation Model			
		ConvNet	ResNet	VGG	AlexNet
Method	Ours	**64.3 ± 0.7**	**46.4 ± 0.6**	**50.3 ± 0.8**	**34.2 ± 2.6**
	DSA	52.1 ± 0.4	42.8 ± 1.0	43.2 ± 0.5	**35.9 ± 1.3**
	KIP	47.6 ± 0.9	36.78 ± 1.0	42.1 ± 0.4	24.4 ± 3.9

Table 3. Despite trained for a specific architecture, our synthetic images do not seem to suffer from much over-fitting to that type of model. This is evaluated on CIFAR-10 with 10 images per class.

both datasets. For CIFAR-10, these distilled images can be seen in Figure 4. CIFAR-100 images are visualized in the supplementary material

In Table 2, we also compare with a recent method KIP [25, 26], where the distilled data is learned with respect to the Neural Tangent Kernel. Because KIP training is agnostic to actual network width, we test their result on both a ConvNet of same width as us and other methods (128) and a ConvNet of larger width (1024) (which is shown in KIP paper [26]). Based on the the infinite-width network limit, KIP may exhibit a gap with practical finite-width networks. Our method does not suffer from this limitation and generally achieves better performance. Over all settings, our method, trained on the 128-width network, outperforms KIP results evaluated on both widths, except for just one setting where KIP is applied on the much wider 1024-width network.

As noted in the previous methods [43], we also see significant diminishing returns when allowing more images in our synthetic datasets. For instance, on CIFAR-10, we see an increase from 46.3% to 65.3% classification accuracy when increasing from 1 to 10 images per class, but only an increase from 65.3% to 71.5% when increasing the number of distilled images per class from 10 to 50.

If we look at the one image per class visualizations in Figure 4 (top), we see very abstract, yet still recognizable, representations of each class. When we limit the task to just one synthetic image per class, the optimization is forced to squeeze as much of the class's distinguishing information as possible into just one sample. When we allow more images in which to disperse the class's information, the optimization has the freedom to spread the class's discriminative features among the multiple samples, resulting in a diverse set of structured images we see in Figure 4 (bottom) (e.g., different types of cars and horses with different poses).

Cross-Architecture Generalization. We also evaluate

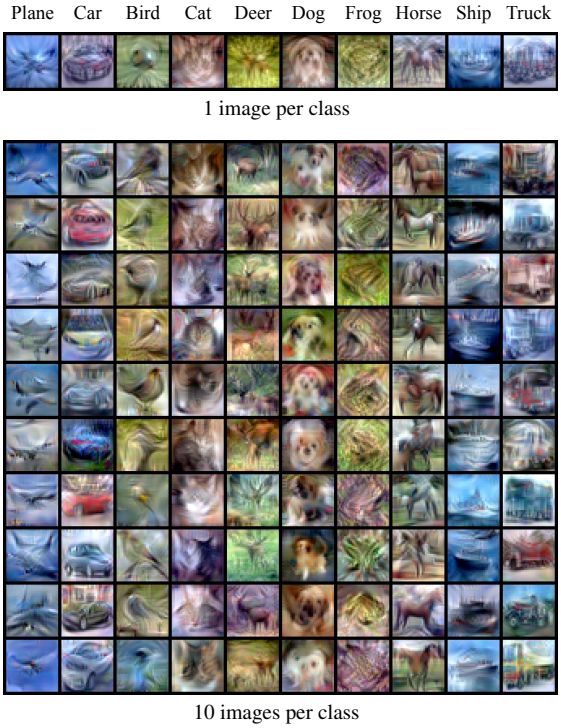


Figure 4. CIFAR-10: The 1 image per class images are more abstract but also more information-dense while the 10 images per class images are more expressive and contain more structure.

how well our synthetic data performs on architectures different from the one used to distill it on the CIFAR-10, 1 image per class task. In Table 3, we show our baseline ConvNet performance and evaluate on ResNet [14], VGG [35], and AlexNet [19]. For KIP, instead of the Kornia [32] ZCA implementation, we use the authors' custom ZCA implementation for evaluation of their method. Our method is solidly the top performer on all the transfer models except for AlexNet where we lie within one standard deviation of DSA. This could simply be attributed to our higher baseline performance, but it still shows that our method is robust to changes in architecture.

4.2. Short-Range vs. Long-Range Matching

Unlike some prior works (DC and DSA), our method performs long-range parameter matching, where N training steps on distilled data match a much larger M steps on real data. Methods that optimizes over entire training processes (e.g., DD and KIP) can be viewed as even longer range matching. However, their performances fall short of our method (e.g., in Table 1), likely due to related instabilities or inexact approximations. Here, we experimentally confirm our hypothesis that long-range matching achieved by larger M and N in our method is superior to the short-range counterparts (such as small M and N and DSA).

In Figure 5 (left), we evaluate our method on different settings of M and N. Really short-range matching (with $N = 1$

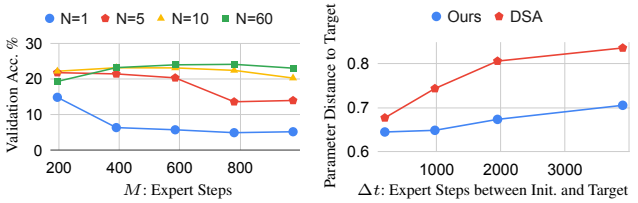


Figure 5. CIFAR-100 (1 image / class). **Left:** Smaller M and N match shorter-range behavior, which performs worse than longer-range matching. **Right:** Over 1000 training steps on distilled data, we track the closest distance in parameter space (normalized MSE in Eqn. 2) to a target set of parameters, obtained with Δ_t training steps on real data. Matching long-range behavior, our method better approximate real data training for longer ranges (large Δ_t).

and small M) generally exhibits worse performance than long-range matching, with the best performance attained when both N and M are relatively large. Furthermore, as we increase N, the power of N combined steps (on distilled data) becomes stronger and can approximate longer-range behavior, leading to the optimal M values shifting to greater values correspondingly.

In Figure 5 (right), we evaluate our method and a short-range matching work (DSA) on their abilities to approximate real training behavior over short and long ranges. Starting from a set of initial parameters, we set the the target parameters to be result of Δ_t training steps on real data (i.e., the long-range behavior that distilled data should mimic). A small (or large) Δ_t means evaluating matching over a short (or long) range. For both methods, we test how close they can train the network (using distilled data) from the same initial parameters to the target parameters. DSA is only optimized to match short-range behavior, and thus error may accumulate during longer training. Indeed, as Δ_t grows larger, DSA fails to mimic the real data behavior over longer ranges. In comparison, our method is optimized for long-range matching and thus performs much better.

4.3. Tiny ImageNet (64×64)

Introduced to the dataset distillation task by the concurrent work, Distribution Matching (DM) [45], we also show the effectiveness of our algorithm on the 200 class, 64×64 Tiny ImageNet [5] dataset (a downscaled subset of ImageNet [6]). To account for the higher image resolution, we move up to a depth-4 ConvNet, similar to DM [45].

Most dataset distillation methods (other than DM) are unable to handle this larger resolution due to extensive memory or time requirement, as the DM authors also observed [45]. In Table 1, our method consistently outperforms the only viable such baseline, DM. Notably, on the 10 image per class task, our method improves the concurrent work DM from 12.9% and 23.2%. A subset of our results is shown in Figure 6. The

[1] Answers for Figure 6: **1st row**: African Elephant, Alps, American Alligator, American Lobster, Camel. **2nd row**: Beaker, Cliff Dwelling, Gazelle, Jellyfish, Kimono. **3rd row**: Lakeside, Lamp, Monarch, Mushroom, (Pipe) Organ. **4th row**: Ox, Pizza, Plate, Pretzel, Snail. **5th row**: Sports Car, Teapot, Teddy, Water Tower, Yorkshire Terrier.



Figure 6. Selected samples distilled from Tiny ImageNet, one image per class. Despite the higher resolution, our method still produces high-fidelity images. (Can you guess which image is for Lamp, Teapot, and Teddy Bear? Check your answers in the footnote![1])

	ImageNette	ImageWoof	ImageFruit	ImageMeow	ImageSquawk	ImageYellow
1 Img/Cls	47.7 ± 0.9	28.6 ± 0.8	26.6 ± 0.8	30.7 ±1.6	39.4 ± 1.5	45.2 ± 0.8
10 Img/Cls	63.0 ± 1.3	35.8 ± 1.8	40.3 ± 1.3	40.4 ± 2.2	52.3 ± 1.0	60.0 ± 1.5
Full Dataset	87.4 ± 1.0	67.0 ± 1.3	63.9 ± 2.0	66.7 ± 1.1	87.5 ± 0.3	84.4 ± 0.6

Table 4. Applying our method to 128×128 resolution ImageNet subsets. On this higher resolution, across various subsets, our method continues to produce high-quality distilled images.

supplementary material contains the rest of the images.

At 200 classes and 64×64 resolution, Tiny ImageNet certainly poses a much harder task than previous datasets. Despite this, many of our distilled images are still recognizable, with a clear color, texture, or shape pattern.

4.4. ImageNet Subsets (128×128)

Next, we push the boundaries of dataset distillation even further by running our method on yet higher resolution images in the form of 128×128 subsets of ImageNet [6]. Again, due to the higher resolution, we increase the depth of our architecture and use a depth-5 ConvNet for the 128×128 ImageNet subsets.

ImageNette (assorted objects) and ImageWoof (dog breeds) are existing subsets [7] designed to be easy and hard to learn respectively. We also introduce ImageFruit (fruits), ImageMeow (cats), ImageSquawk (birds), and ImageYellow (yellow-ish things) to further illustrate our algorithm.

Similar to Tiny ImageNet, most dataset distillation baselines do not scale up to our ImageNet subset settings. As the code of DM [45] is not publicly available now, we choose to only compare to the networks trained on full dataset. We wish to show that our method transfers well to large images and still produces meaningful results at a higher resolution. Validation set accuracies are presented in Table 4.

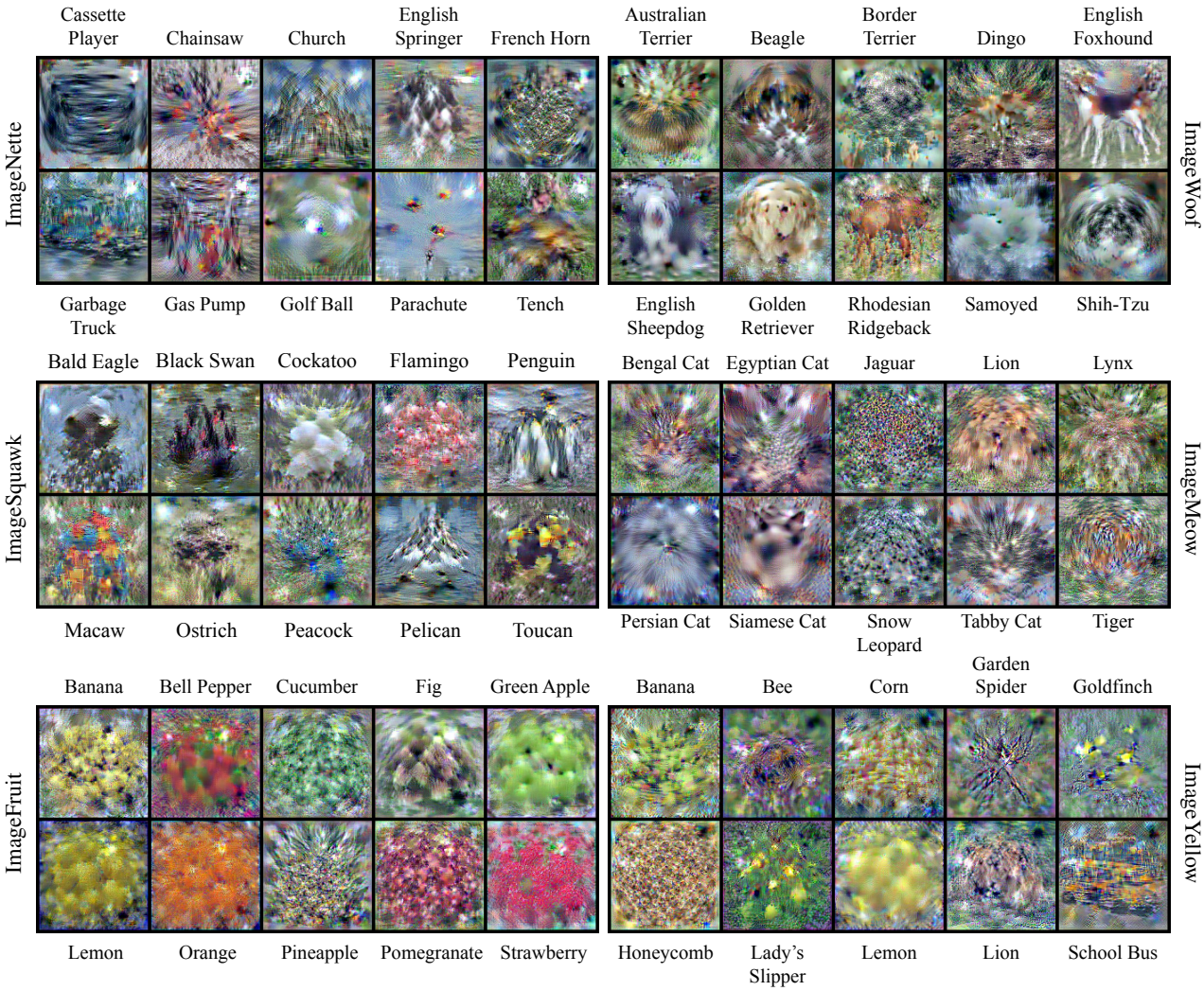


Figure 7. Our method is the first capable of distilling higher-resolution (128×128) images, allowing us to explore the ImageNet [6] dataset.

While all of the generated images are devoid of high-frequency noise, the tasks still differ in the type of distilled image they induce. For tasks where all the classes have similar structure but unique textures like ImageSquawk (Figure 7), the distilled images may not have much structure but instead store discriminating information in the textures.

Conversely, for tasks where all classes have similar color or textures like ImageYellow (Figure 7), the distilled images seem to diverge from their common trait and accentuate the structure or secondary color that makes them unique. Specifically, note the differences between the distilled "Banana" images for the ImageFruit and ImageYellow (bottom row, Figure 7). Although the expert trajectory-generating networks saw the same "Banana" training images, the distilled images differ between the two tasks. The distilled "Banana" for the ImageYellow task is clearly much "greener" than the equivalent image for the ImageFruit task. This implies that the expert networks identify different features by which to identify classes based on the other classes in the task.

5. Discussion

In this work, we introduced a new method of dataset distillation by means of directly optimizing the synthetic data to induce similar network training dynamics as the real data. Method-wise, the main difference between ours and prior approaches is that we are neither limited to the short-range single-step matching nor subject to instability and compute intensity of optimizing over the full training process. Our method nicely balances these two regimes and shows superior performance over both. Additionally, our use of pre-computed trajectories allows for significant memory saving, at the cost of disk storage. Unlike prior methods, ours is the first to scale to higher-resolution data, such as 128×128 ImageNet images. Scaling to such larger datasets not only allows us to gain interesting insights of the dataset (e.g., in Section 4.4) but also may serve as an important step towards practical applications of dataset distillation on real-world datasets.

6. Acknowledgements

This material is based upon work supported by the National Science Foundation Graduate Research Fellowship under Grant No. DGE1745016.

References

[1] Olivier Bachem, Mario Lucic, and Andreas Krause. Practical coreset constructions for machine learning. *arXiv preprint arXiv:1703.06476*, 2017. 3

[2] Ondrej Bohdal, Yongxin Yang, and Timothy Hospedales. Flexible dataset distillation: Learn labels instead of images. *arXiv preprint arXiv:2006.08572*, 2020. 1, 2, 5

[3] Zalán Borsos, Mojmír Mutnỳ, and Andreas Krause. Coresets via bilevel optimization for continual learning and streaming. *arXiv preprint arXiv:2006.03875*, 2020. 3

[4] Yutian Chen, Max Welling, and Alex Smola. Super-samples from kernel herding. In *UAI*, 2010. 3, 5

[5] cs231n.stanford.edu. Cs231n: Convolutional neural networks for visual recognition. 7

[6] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009. 2, 7, 8

[7] Fastai. Fastai/imagenette: A smaller subset of 10 easily classified classes from imagenet, and a little more french. 7

[8] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning*, pages 1126–1135. PMLR, 2017. 2

[9] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020. 3

[10] Spyros Gidaris and Nikos Komodakis. Dynamic few-shot visual learning without forgetting. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4367–4375, 2018. 5

[11] Jack Goetz and Ambuj Tewari. Federated learning via synthetic data. *arXiv preprint arXiv:2008.04489*, 2020. 2

[12] Caglar Gulcehre, Ziyu Wang, Alexander Novikov, Tom Le Paine, Sergio Gomez Colmenarejo, Konrad Zolna, Rishabh Agarwal, Josh Merel, Daniel Mankowitz, Cosmin Paduraru, et al. Rl unplugged: Benchmarks for offline reinforcement learning. *arXiv e-prints*, pages arXiv–2006, 2020. 3

[13] Sariel Har-Peled and Akash Kushal. Smaller coresets for k-median and k-means clustering. *Discrete & Computational Geometry*, 37(1):3–19, 2007. 3

[14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. 6

[15] Geoffrey Hinton, Oriol Vinyals, and Jeffrey Dean. Distilling the knowledge in a neural network. In *NIPS Deep Learning and Representation Learning Workshop*, 2015. 1

[16] Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. *Advances in neural information processing systems*, 29:4565–4573, 2016. 3

[17] Tero Karras, Miika Aittala, Janne Hellsten, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Training generative adversarial networks with limited data. In *NeurIPS*, 2020. 4

[18] Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. Technical report, Citeseer, 2009. 5

[19] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25:1097–1105, 2012. 5, 6

[20] Agata Lapedriza, Hamed Pirsiavash, Zoya Bylinskii, and Antonio Torralba. Are all training examples equally valuable? *arXiv preprint arXiv:1311.6510*, 2013. 3

[21] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 5

[22] Guang Li, Ren Togo, Takahiro Ogawa, and Miki Haseyama. Soft-label anonymous gastric x-ray image distillation. In *ICIP*, 2020. 2

[23] Dougal Maclaurin, David Duvenaud, and Ryan Adams. Gradient-based hyperparameter optimization through reversible learning. In *ICML*, 2015. 2

[24] Wojciech Masarczyk and Ivona Tautkute. Reducing catastrophic forgetting with learning on synthetic data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 252–253, 2020. 3

[25] Timothy Nguyen, Zhourong Chen, and Jaehoon Lee. Dataset meta-learning from kernel ridge-regression. In *International Conference on Learning Representations*, 2020. 1, 2, 5, 6

[26] Timothy Nguyen, Roman Novak, Lechao Xiao, and Jaehoon Lee. Dataset distillation with infinitely wide convolutional networks. *arXiv preprint arXiv:2107.13034*, 2021. 1, 2, 5, 6

[27] Alex Nichol, Joshua Achiam, and John Schulman. On first-order meta-learning algorithms. *arXiv preprint arXiv:1803.02999*, 2018. 2

[28] J Arturo Olvera-López, J Ariel Carrasco-Ochoa, J Francisco Martínez-Trinidad, and Josef Kittler. A review of instance selection methods. *Artificial Intelligence Review*, 34(2):133–143, 2010. 3

[29] Takayuki Osa, Joni Pajarinen, Gerhard Neumann, J Andrew Bagnell, Pieter Abbeel, and Jan Peters. An algorithmic perspective on imitation learning. *arXiv preprint arXiv:1811.06711*, 2018. 3

[30] Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. Deepmimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions on Graphics (TOG)*, 37(4):1–14, 2018. 3

[31] Xue Bin Peng, Ze Ma, Pieter Abbeel, Sergey Levine, and Angjoo Kanazawa. Amp: Adversarial motion priors for stylized physics-based character control. *arXiv preprint arXiv:2104.02180*, 2021. 3

[32] E. Riba, D. Mishkin, D. Ponsa, E. Rublee, and G. Bradski. Kornia: an open source differentiable computer vision library for pytorch. In *Winter Conference on Applications of Computer Vision*, 2020. 5, 6

[33] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011. 3

[34] Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach. In *ICLR*, 2018. 3

[35] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014. 6

[36] Felipe Petroski Such, Aditya Rawal, Joel Lehman, Kenneth

Stanley, and Jeffrey Clune. Generative teaching networks: Accelerating neural architecture search by learning to generate synthetic training data. In *International Conference on Machine Learning*, pages 9206–9216. PMLR, 2020. 3

[37] Ilia Sucholutsky and Matthias Schonlau. Secdd: Efficient and secure method for remotely training neural networks. *arXiv preprint arXiv:2009.09155*, 2020. 2

[38] Ilia Sucholutsky and Matthias Schonlau. Soft-label dataset distillation and text dataset distillation. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2021. 1, 2

[39] Mariya Toneva, Alessandro Sordoni, Remi Tachet des Combes, Adam Trischler, Yoshua Bengio, and Geoffrey J Gordon. An empirical study of example forgetting during deep neural network learning. *arXiv preprint arXiv:1812.05159*, 2018. 3, 5

[40] Ngoc-Trung Tran, Viet-Hung Tran, Ngoc-Bao Nguyen, Trung-Kien Nguyen, and Ngai-Man Cheung. Towards good practices for data augmentation in gan training. *arXiv preprint arXiv:2006.05338*, 2, 2020. 4

[41] Ivor W Tsang, James T Kwok, and Pak-Ming Cheung. Core vector machines: Fast svm training on very large data sets. *JMLR*, 6(Apr):363–392, 2005. 3

[42] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022*, 2016. 5

[43] Tongzhou Wang, Jun-Yan Zhu, Antonio Torralba, and Alexei A Efros. Dataset distillation. *arXiv preprint arXiv:1811.10959*, 2018. 1, 2, 5, 6

[44] Bo Zhao and Hakan Bilen. Dataset condensation with differentiable siamese augmentation. *arXiv preprint arXiv:2102.08259*, 2021. 1, 2, 3, 4, 5

[45] Bo Zhao and Hakan Bilen. Dataset condensation with distribution matching. *arXiv preprint arXiv:2110.04181*, 2021. 1, 2, 3, 5, 7

[46] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. Dataset condensation with gradient matching. In *International Conference on Learning Representations*, 2020. 1, 2, 3, 5

[47] Shengyu Zhao, Zhijian Liu, Ji Lin, Jun-Yan Zhu, and Song Han. Differentiable augmentation for data-efficient gan training. In *NeurIPS*, 2020. 4

[48] Zhengli Zhao, Zizhao Zhang, Ting Chen, Sameer Singh, and Han Zhang. Image augmentations for gan training. *arXiv preprint arXiv:2006.02595*, 2020. 4

[49] Yanlin Zhou, George Pu, Xiyao Ma, Xiaolin Li, and Dapeng Wu. Distilled one-shot federated learning. *arXiv preprint arXiv:2009.07999*, 2020. 2

A. Appendix

A.1. Additional Visualizations

We first include some additional visualizations here. CIFAR-100 (1 image per class) can be seen in Figure 10. All of Tiny ImageNet (1 image per class) is broken up into Figures 17 and 18. We specifically show the 10 best and worst-performing distilled classes in Figures 11 and 12 respectively. We include 10 image per class visualizations of all our 128×128 ImageNet subsets in Figures 19-24.

A.2. Additional Quantitative Results

Analysis of learned learning rates α. In Figure 9, we explore the effect of our learnable synthetic step size α. The left plot confirms that we learn different values of α for different combinations of M and N. The logic here is that different numbers of synthetic steps N require a different step size α to cover the same distance as M real steps. The right plot illustrates the practical benefits of our adaptive learning rate; instead of yet another hyper-parameter to tune, our adaptive learning rate works from a wide range of initializations.

A.2.1 Additional Ablation Studies

Initialization, normalization, and augmentation. In the main paper, we show ablations over several hyper-parameters. Here, we study the role of initialization, data normalization, and data augmentation for CIFAR-100 (1 image per class) in Table 5. For initialization in particular, recall that we typically initilizae our synthetic images with real samples. Here, we evaluate initializing with Gaussian noise instead. Visualizations of these distilled sets can be seen in Figures 13-15. We also include a visualization of a set distilled with only one expert trajectory in Figure 16.

Setting	Ours	Gauss. Init.	w/o ZCA	w/o Diff. Aug.
Acc.	24.3 ± 0.3	22.9 ± 0.4	19.1 ± 0.6	20.7 ± 0.7

Table 5. As we ablate our categorical hyper-parameters, we still achieve state-of-the-art performance (compared to DSA: 13.9%). This is evaluated on CIFAR-100 with 1 image per class. Each design choice in our final method improves the performance of distilled images. Here we use the default set of hyper-parameters for these ablations.

Performance w.r.t. the number of expert trajectories. Since they effectively make up our method's "training set," it is reasonable to assume that having more expert trajectories would lead to better performance. We see that this is indeed the case for the CIFAR-100, 1 image per class setting in Figure 8 (left). However, what's most interesting is the sharp, logarithmic increase in validation accuracy w.r.t. the number of experts. We note the most amount of improvement when increasing from 1 to 20 experts but see almost complete saturation by the time we reach 200. Given how high-dimensional the parameter space of a neural network is, it is remarkable that we can achieve such high performance with so few expert trajectories.

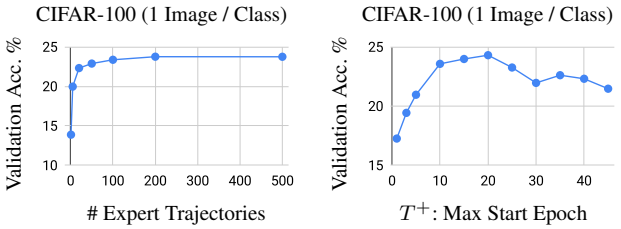


Figure 8. **Left:** We see logarithmic performance improvement with respect to the number of expert trajectories used, quickly saturating near 200. **Right:** The upper bound on the expert epoch at which the synthetic data starts working cannot be too high or low to ensure quality learning signal.

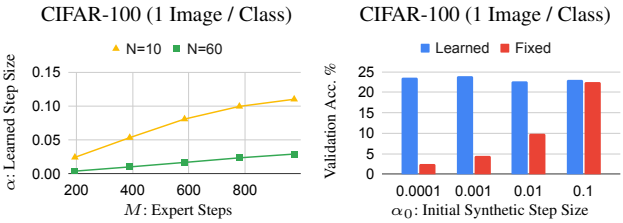


Figure 9. **Left:** Our learned synthetic step size α seems to scale inversely with the number of synthetic steps N to cover the same distance in parameter space as the expert steps M. **Right:** Having a learnable step size α saves us from having to search for an appropriate fixed α_0.

Performance w.r.t. expert time-step range. When we initialize our student networks, we do so at a randomly selected time-step from an expert trajectory. We find that it is important to put an upper bound on this starting time-step (Figure 8, right). If the upper bound is too high, the synthetic data receives gradients from points where the experts movements are small and uninformative. If it is too low, the synthetic data is never exposed to mid and later points in the trajectories, missing out on a significant portion of the training dynamics.

A.3. Experiment Details

Hyper-Parameters. In Table 6, we enumerate the hyper-parameters used for our results reported in the main text. Limited compute forced us to batch our synthetic data for some of the larger sets. The "ConvNet" architectures are as explained in the main text.

Compute resources. We had a relatively limited compute budget for our experiments, using any GPU we could access. As such, our experiments were run on a mixture of RTX2080ti, RTX3090, and RTX6000 GPUs. The largest amount of VRAM we used for a single experiment was 144GB over 6xRTX6000 GPUs.

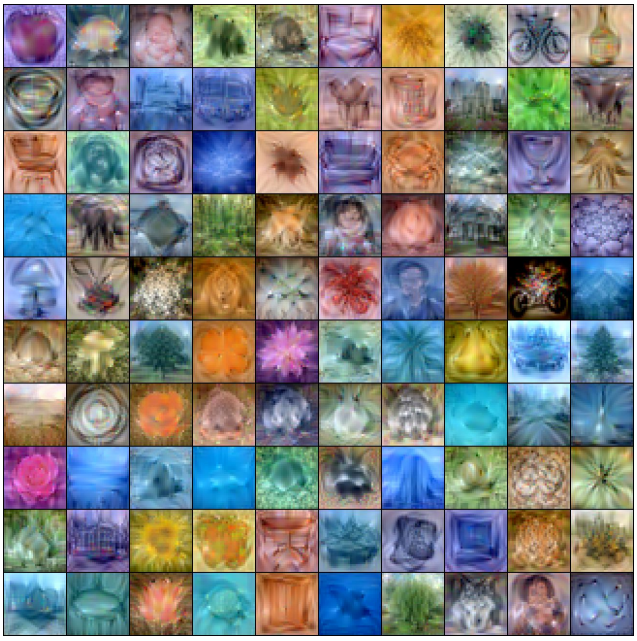


Figure 10. CIFAR-100, 1 Image Per Class

Dataset	Model	Img/Cls	Synthetic Steps (N)	Expert Epochs ($M^\dagger$)	Max Start Epoch (T^+)	Synthetic Batch Size ($\|b\|$)
CIFAR-10	ConvNetD3	1	50	2	2	-
		10	30	2	20	-
		50	30	2	20	100
CIFAR-100	ConvNetD3	1	20	3	20	-
		10	20	2	20	-
		50	80	2	20	125
Tiny ImageNet	ConvNetD4	1	10	2	10	-
		10	20	2	40	200
		50	20	2	40	300
ImageNet (All)	ConvNetD5	1	20	2	10	-
		10	20	2	10	20

Table 6. Hyper-parameters used for our distillation experiments. A synthetic batch size of "-" indicates that we used the full support set at each synthetic step. Note: instead of number of expert *updates* (M), here we list number of expert *epochs* ($M^\dagger$) for simplicity across datasets.

Training Time. Distillation time varied based on dataset and type and number of GPUs used. Regardless of dataset or compute resources, time per distillation iteration scaled linearly with the number of synthetic steps N. For CIFAR-100, 1 image per class with $N = 20$, we had an average time of 0.6 seconds per distillation step when using a single RTX3090. We ran our experiments for 5000 distillation steps but saw the most (if not all) improvement within the first 1000.

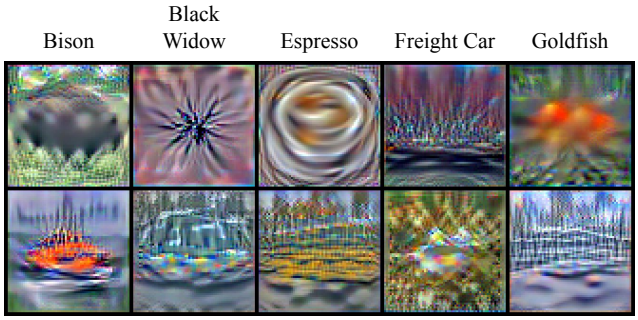


Figure 11. Most-correct distilled images for Tiny ImageNet ($\geq 30\%$)

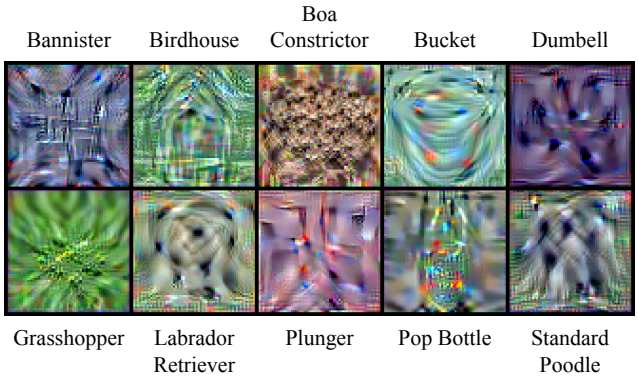


Figure 12. Least-correct distilled images for Tiny ImageNet ($\leq 4\%$)

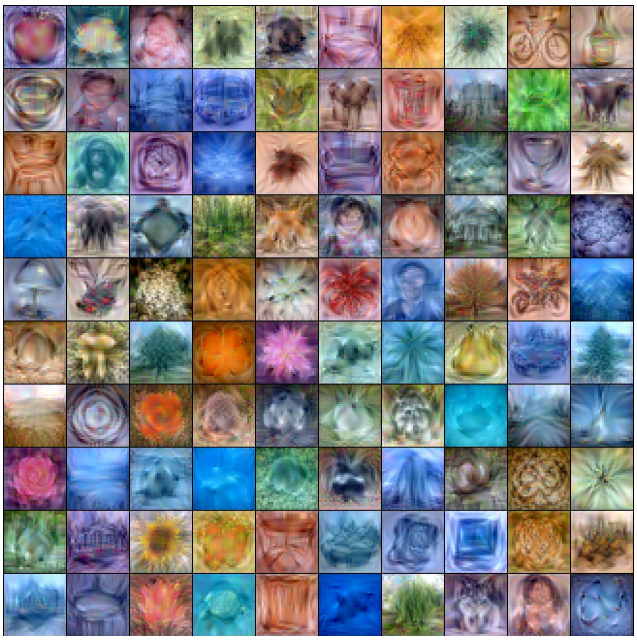


Figure 13. CIFAR-100, Initialized as Random Noise

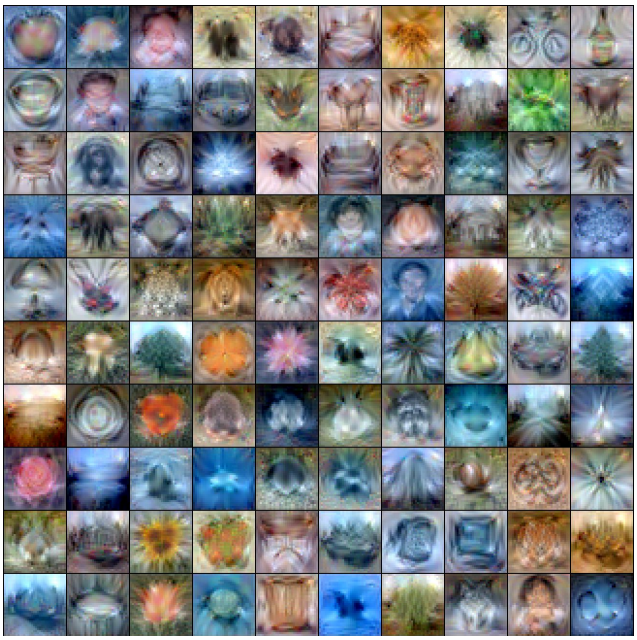


Figure 15. CIFAR-100, No Differentiable Augmentation

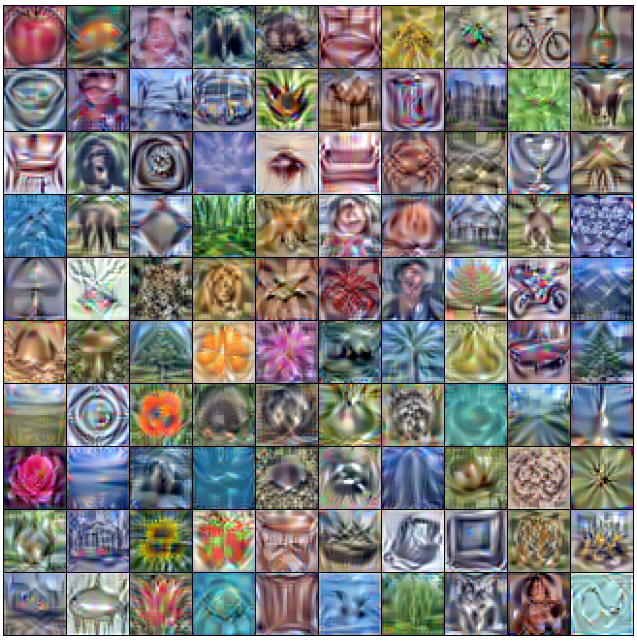


Figure 14. CIFAR-100, No ZCA Whitening

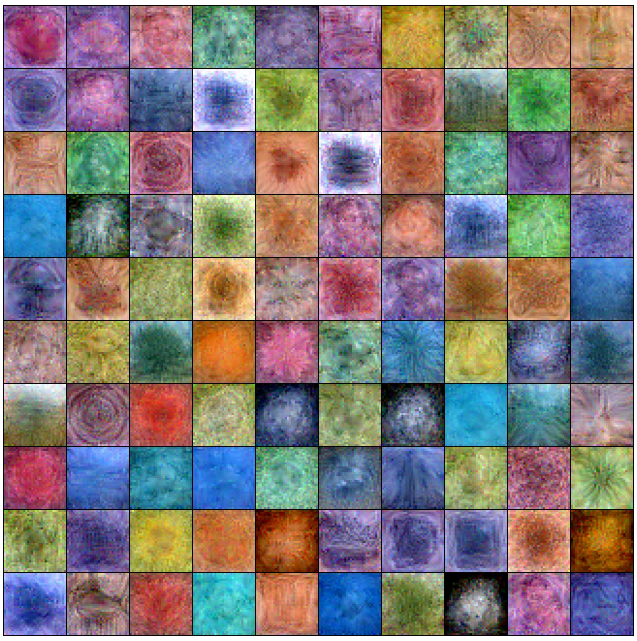


Figure 16. CIFAR-100, Only 1 Expert Trajectory



Figure 17. Tiny ImageNet, 1 Image Per Class (Classes 1-100)



Figure 18. Tiny ImageNet, 1 Image Per Class (Classes 101-200)



Figure 19. ImageNette, 10 Images Per Class



Figure 20. ImageWoof, 10 Images Per Class



Figure 21. ImageSquawk, 10 Images Per Class



Figure 22. ImageMeow, 10 Images Per Class

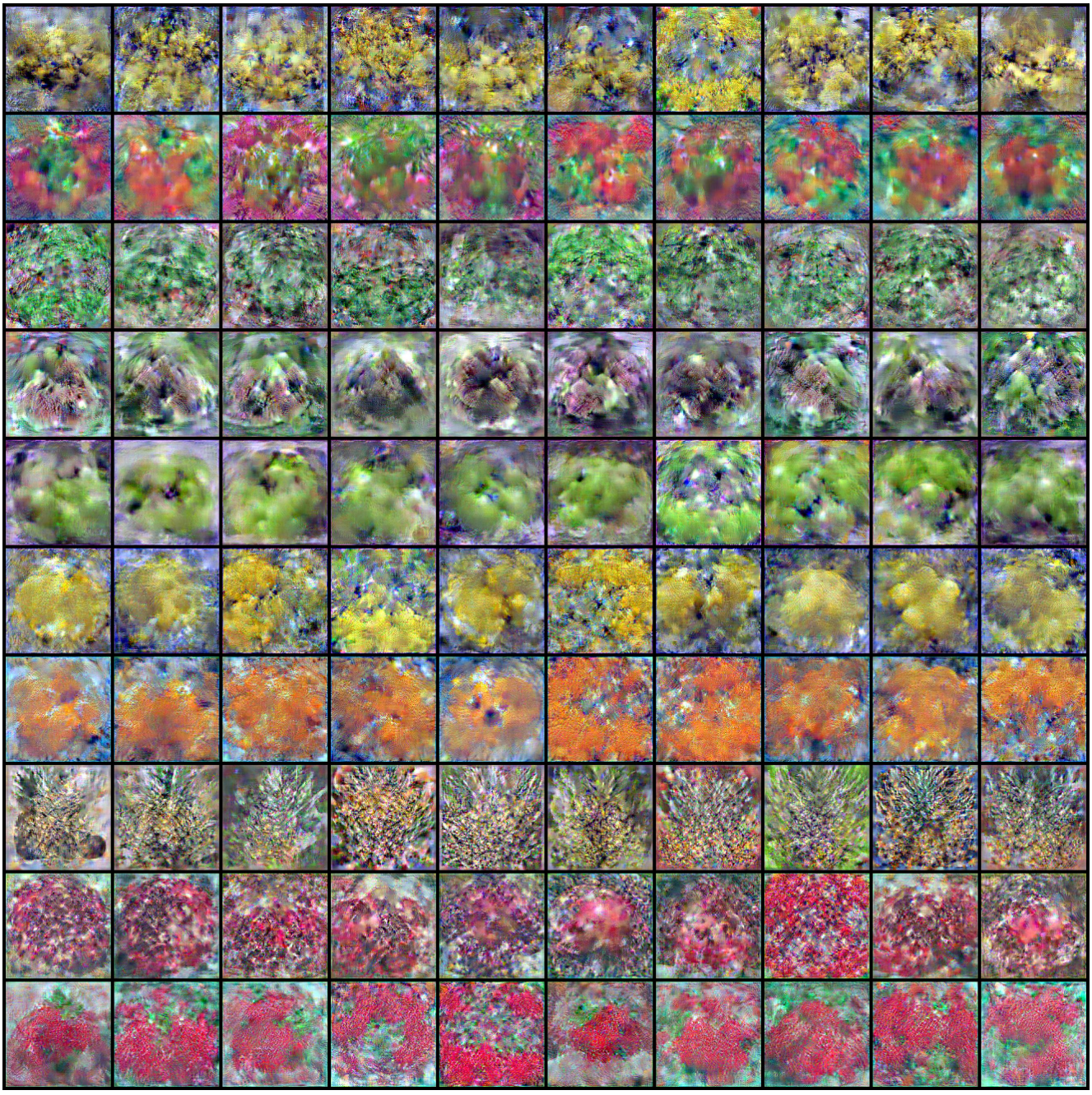


Figure 23. ImageFruit, 10 Images Per Class

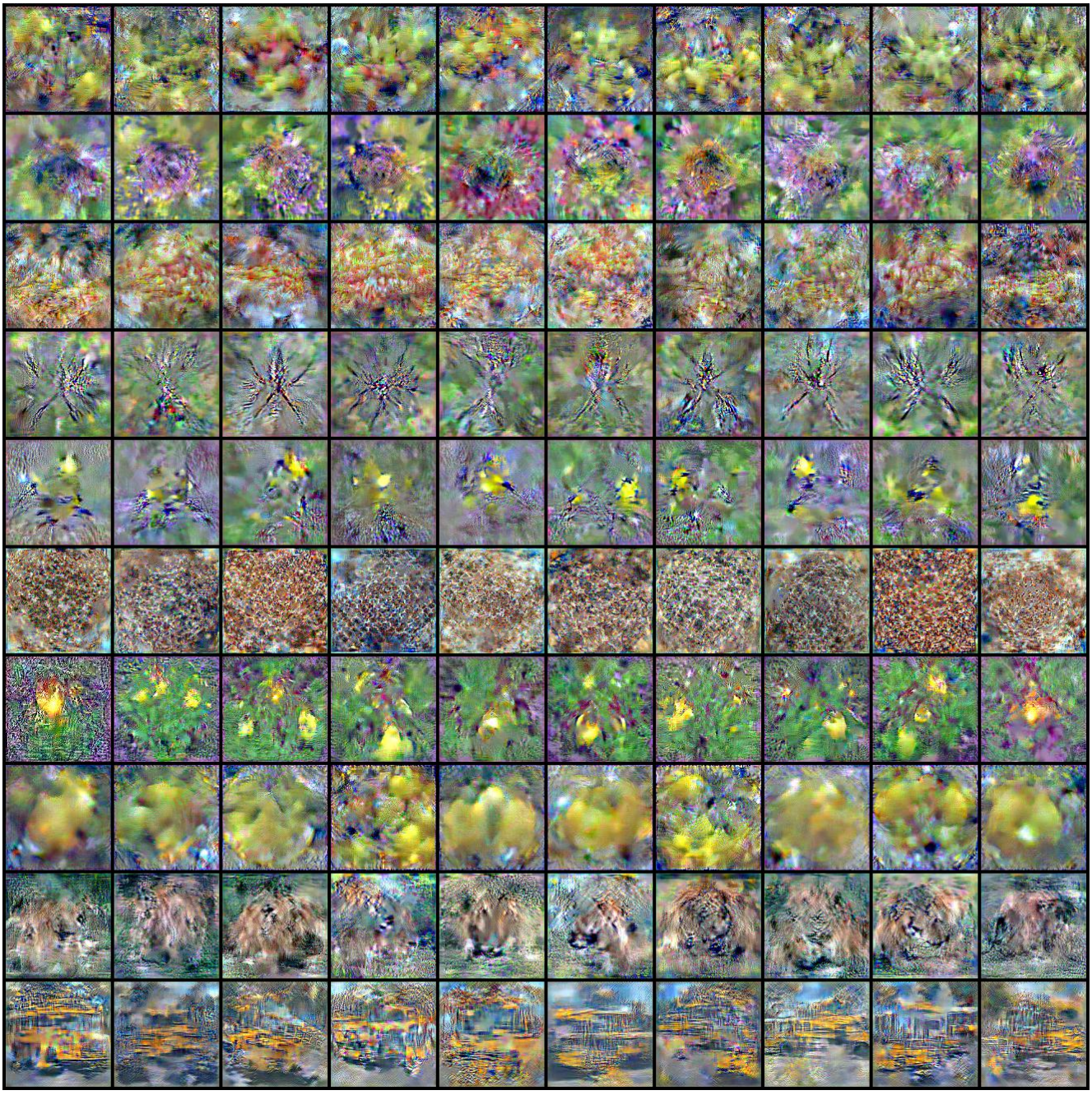


Figure 24. ImageYellow, 10 Images Per Class